Status	Finished
Started	Friday, 9 October 2026, 12:11 PM
Completed	Friday, 9 October 2026, 12:15 PM
Duration	4 mins 21 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
def delete_line(filename, line_number):
    # Read all lines from the source file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Exclude the line corresponding to the specified line number (1-indexed)
    updated_lines = [line for idx, line in enumerate(lines, start=1) if idx != line_number]

    # Write the remaining lines to 'output.txt'
    with open('output.txt', 'w') as file:
        file.writelines(updated_lines)

if __name__ == "__main__":
    try:
        # Read the file name from standard input
        filename = input().strip()

        # Read the line number to delete
        line_num = int(input().strip())

        # Execute the deletion function
        delete_line(filename, line_num)
    except (IOError, ValueError):
        pass

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
def count_words_in_file(filename):
    # Open and read the contents of the text file
    with open(filename, 'r') as file:
        text = file.read()
        # Split text by any whitespace to isolate individual words
        words = text.split()
        # Print the total count matching the expected format
        print(f"Total words: {len(words)}")

if __name__ == "__main__":
    try:
        # Read the file name from standard input
        filename = input().strip()
        count_words_in_file(filename)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
def insert_line_at_position(filename, position, new_content):
    # 1. Read all existing lines from the input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Ensure the new content ends with a newline character
    if not new_content.endswith('\n'):
        new_content += '\n'

    # 2. Insert the new line using 0-based indexing (position - 1)
    # If the file lacks trailing newlines, adjust the preceding line
    idx = position - 1
    if idx > 0 and idx - 1 < len(lines) and not lines[idx - 1].endswith('\n'):
        lines[idx - 1] += '\n'

    lines.insert(idx, new_content)

    # 3. Write the updated sequence to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)


if __name__ == "__main__":
    try:
        # Read the file name, position, and new line contents from standard input
        filename = input().strip()
        position = int(input().strip())
```

```
        new_content = input()

        insert_line_at_position(filename, position, new_content)
    except (IOError, ValueError):
        pass

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import string

def find_and_save_palindromes(filename):
    # Open and read the input file
    with open(filename, 'r') as file:
        text = file.read()

    # Split text into words/tokens by whitespace
    words = text.split()
    palindromes = []
    seen = set()

    for word in words:
        # Strip punctuation and convert to lowercase for uniform comparison
        cleaned = word.strip(string.punctuation).lower()

        # Check if the cleaned word is a valid palindrome
        if cleaned and cleaned == cleaned[::-1]:
            if cleaned not in seen:
                seen.add(cleaned)
                palindromes.append(cleaned)

    # Write the identified palindromes to output.txt
    with open('output.txt', 'w') as outfile:
        for p in palindromes:
            outfile.write(p + '\n')

if __name__ == "__main__":
    try:
        # Read the file name from standard input
        filename = input().strip()
        find_and_save_palindromes(filename)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	output1.txt
		This is the source file.
		It contains multiple lines of text.
		Here is another line.

Answer: (penalty regime: 0 %)

```
def copy_file_contents(source_filename, dest_filename):
    # Open the source file in read mode and destination file in write mode
    with open(source_filename, 'r') as src_file:
        content = src_file.read()

    with open(dest_filename, 'w') as dest_file:
        dest_file.write(content)

if __name__ == "__main__":
    try:
        # Read the source and destination file names from standard input
        source_file = input().strip()
        dest_file = input().strip()

        copy_file_contents(source_file, dest_file)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		output1.txt
			This is the source file.
			It contains multiple lines of text.
			Here is another line.
				This is the source file.
				It contains multiple lines of text.
				Here is another line.
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		output2.txt
			Hello, world!
			Python programming is amazing.
			Let's copy this text to another file.
				Hello, world!
				Python programming is amazing.
				Let's copy this text to another file.
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input3.txt
		output3.txt
			Single line.	Single line.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
def read_specific_line(filename, line_number):
    # Open and read all lines from the file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Check if the line number is within the valid range
    if 1 <= line_number <= len(lines):
        # Print the requested line and remove trailing newlines
        print(lines[line_number - 1].strip())

if __name__ == "__main__":
    try:
        # Read the file name from standard input
        filename = input().strip()

        # Read the line number to read
        line_num = int(input().strip())

        # Read and print the specific line
        read_specific_line(filename, line_num)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import string
from collections import Counter

def process_word_frequencies():
    # 1. Read the raw text directly from standard input
    input_text = sys.stdin.read()

    # 2. Split the text into individual words
    raw_words = input_text.split()
    cleaned_words = []

    for word in raw_words:
        # Strip trailing/leading punctuation and normalize to lowercase
        cleaned = word.strip(string.punctuation).lower()
        if cleaned:  # Only add if it's not an empty string
            cleaned_words.append(cleaned)

    # 3. Calculate word frequencies
    frequencies = Counter(cleaned_words)

    # 4. Sort words alphabetically
    sorted_words = sorted(frequencies.keys())

    # 5. Write the correctly normalized counts to output.txt
    with open('output.txt', 'w') as out_file:
        for word in sorted_words:
            out_file.write(f"{word}: {frequencies[word]}\n")

if __name__ == "__main__":
    process_word_frequencies()

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import string

def find_longest_word(filename):
    # Open and read the contents of the text file
    with open(filename, 'r') as file:
        text = file.read()

    # Split text into individual words and strip surrounding punctuation
    words = text.split()
    cleaned_words = [word.strip(string.punctuation) for word in words if word.strip(string.punctu

    if cleaned_words:
        # Find the word with the maximum length
        longest_word = max(cleaned_words, key=len)
        print(f"Longest word: {longest_word}")

if __name__ == "__main__":
    try:
        # Read the file name from standard input
        filename = input().strip()
        find_longest_word(filename)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
		Line one.
		Line two.
		eerht eniL.
		Line four.

Answer: (penalty regime: 0 %)

```
import string

def reverse_line_preserving_punctuation(filename, line_number):
    # 1. Read all lines from the input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    idx = line_number - 1
    if 0 <= idx < len(lines):
        original_line = lines[idx]

        # Strip trailing newline for clean processing
        has_newline = original_line.endswith('\n')
        line_content = original_line.rstrip('\n')

        # Check if the line ends with a common punctuation mark (. ! ?)
        has_punctuation = False
        punctuation_char = ""
        if line_content and line_content[-1] in string.punctuation:
            has_punctuation = True
            punctuation_char = line_content[-1]
            line_content = line_content[:-1]  # Remove it temporarily for reversal

        # Reverse the text characters
        reversed_text = line_content[::-1]

        # Reconstruct the line: add punctuation back to the end, then newline
        if has_punctuation:
            reversed_text += punctuation_char
        if has_newline:
            reversed_text += '\n'

```

```
        lines[idx] = reversed_text

    # 3. Write the updated content to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == "__main__":
    try:
        filename = input().strip()
        line_num = int(input().strip())
        reverse_line_preserving_punctuation(filename, line_num)
    except (IOError, ValueError, FileNotFoundError):
        pass
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
def replace_specific_line(filename, line_number, new_content):
    # 1. Read all existing lines from the input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Ensure the new content ends with a newline character
    if not new_content.endswith('\n'):
        new_content += '\n'

    # 2. Replace the line using 0-based indexing (position - 1)
    idx = line_number - 1
    if 0 <= idx < len(lines):
        lines[idx] = new_content

    # 3. Write the updated sequence to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == "__main__":
    try:
        # Read the file name, position, and new line contents from standard input
        filename = input().strip()
        line_num = int(input().strip())
        new_content = input()

        replace_specific_line(filename, line_num, new_content)
    except (IOError, ValueError, FileNotFoundError):
        pass

```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`2`
		`Updated line two.`
			`Line one.`
			`Updated line two.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Updated line two.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`2`
		`Line B Updated.`
			`Line A.`
			`Line B Updated.`
			`Line C.`
				`Line A.`
				`Line B Updated.`
				`Line C.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.